Task Manager Assessment

Flowboard

A Linear-inspired Kanban board with private guest workspaces, built on React, TypeScript and Supabase.

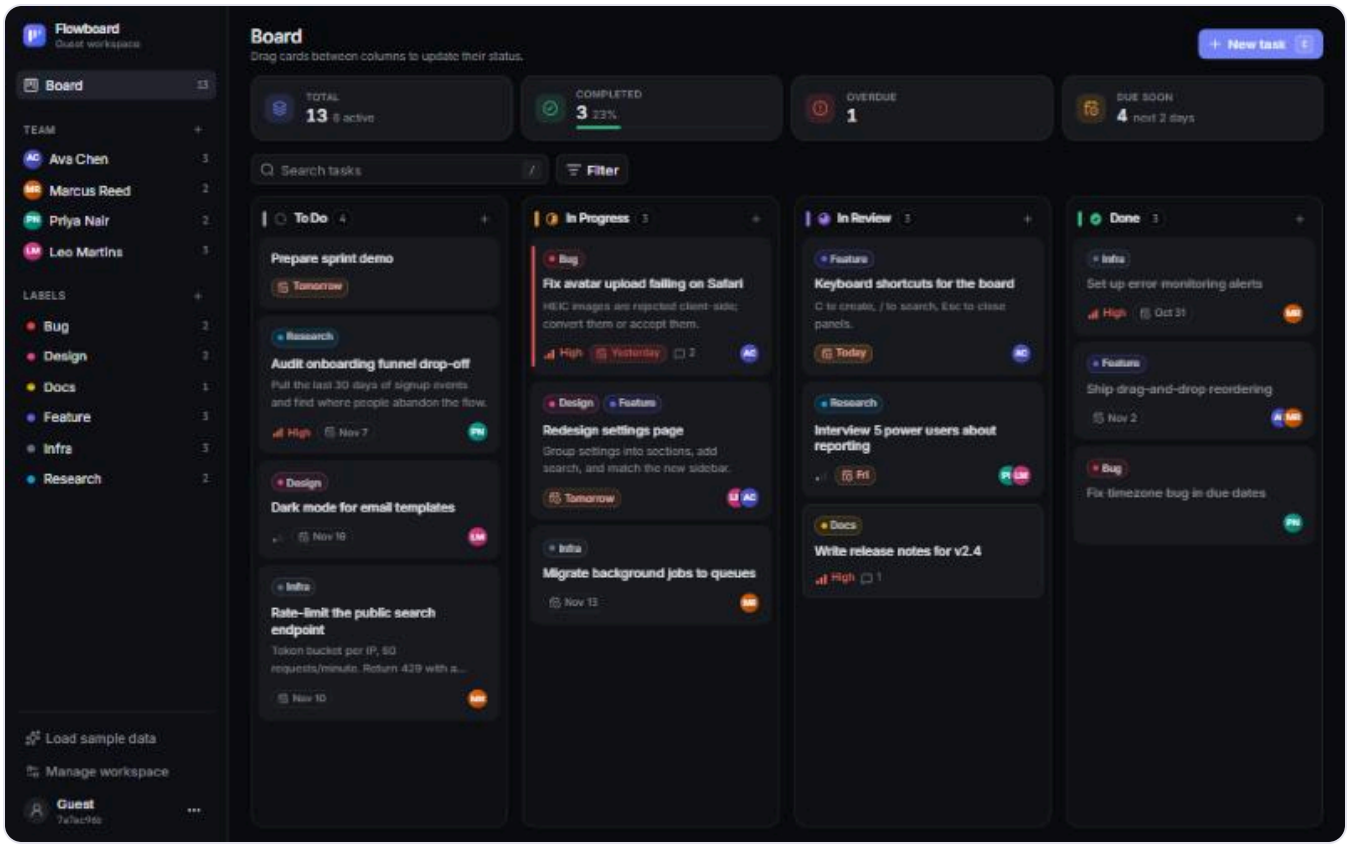
Author Matthew Feyler

Date November 4, 2026

Stack React 19 · TypeScript · Vite · Tailwind CSS v4 · shadcn/ui (Radix) · dnd-kit · Framer Motion · TanStack Query · Supabase · Cloudflare Pages

LIVE DEMO
flowboard-btz.pages.dev

GITHUB REPOSITORY
github.com/MonklyS/flowboard



The board in dark mode with the sample data loaded: status-colored columns, labeled cards, priority and due-date badges, assignee avatars and the summary strip.

01 Overview

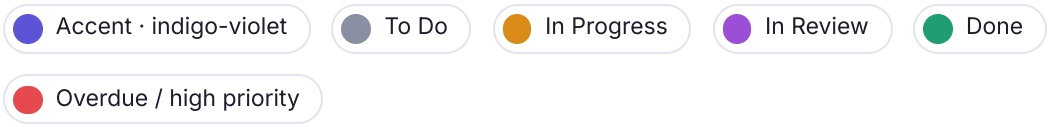
Flowboard is a single-page web app where each visitor gets a private board with four columns: **To Do**, **In Progress**, **In Review** and **Done**. Tasks can be created with a title, description, priority, due date, assignees and labels, then dragged between and within columns. Every change persists to Supabase immediately. On first launch the app creates a Supabase **anonymous** user automatically, so there is no sign-up. Row Level Security ensures each guest can only ever read or write their own rows.

Beyond the required board, all seven suggested advanced features are implemented: team members and multi-assignees, comments, an activity log, labels, due-date urgency indicators, search and filters, and a board summary. The app also has realtime sync across tabs, light/dark themes, keyboard shortcuts and a responsive mobile layout.

02 Design decisions

Visual system

The look follows Linear and Asana: calm neutrals, one accent color, dense but readable cards, and color used for meaning rather than decoration. All colors are OKLCH design tokens defined once in CSS and mapped into Tailwind, so light and dark themes are the same tokens with different values.



- **Typography:** Inter Variable for the UI (with its alternate glyph sets for a crisper look), JetBrains Mono for counts and IDs, and tabular numerals so numbers don't shift.
- **Hierarchy:** columns are recessed, tinted surfaces with a colored status bar, a status glyph and a count. Cards are raised surfaces with a hairline shadow. Inside a card, labels come first, then a bold title, a muted two-line description, and a quiet metadata row (priority, due badge, comments, avatars).
- **Status color as a system:** each status hue is reused in the column header, the progress-style status icon (an empty ring that fills to a check), activity entries and the stats strip.
- **Components:** shadcn/ui on Radix primitives gives accessible dialogs, popovers, command menus and sheets, all restyled with the tokens. Icons are Lucide.

Drag and drop

- **dnd-kit** with three sensors: mouse (6px threshold, so a click still opens the card), touch (long-press, so the board still scrolls on phones) and keyboard (`Space` to lift and drop, arrow keys to move, `Esc` to cancel), plus screen-reader announcements.
- A custom collision strategy prefers whatever is under the pointer and snaps to the nearest card in that column, which keeps cross-column moves predictable. The dragged card floats in an overlay with a slight tilt and lift shadow, and a dashed placeholder shows where it will land.

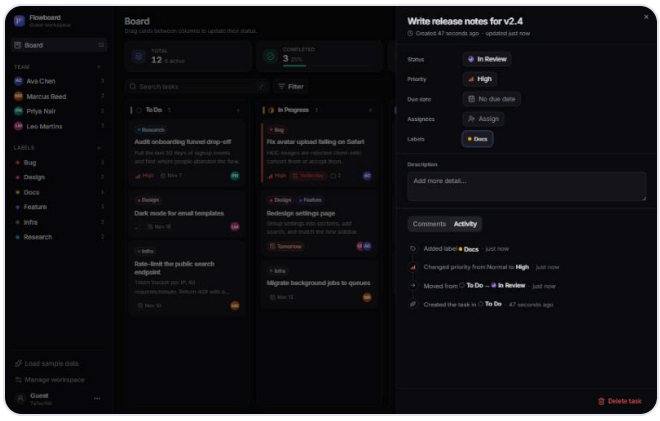
- **Ordering** uses fractional positions: a dropped card takes the midpoint between its neighbors, so a move writes exactly one row.
- On drop the query cache is patched *synchronously* before the network request, so the card never flickers back. If the request fails the cache rolls back and a toast explains what happened.

Data and architecture

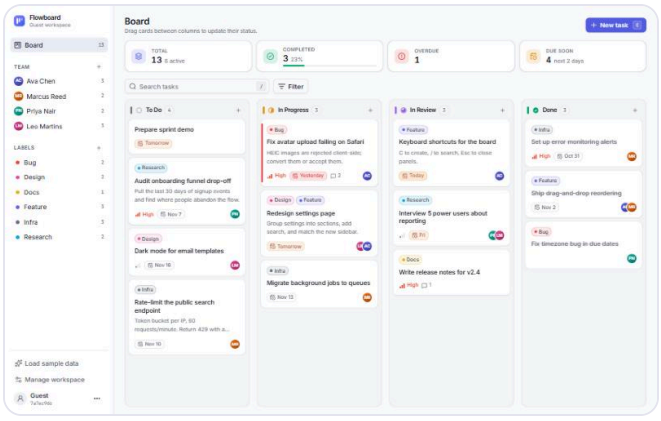
- **Supabase is called directly from the browser**, with no custom API. RLS is the security boundary either way, and skipping a middle tier removes a deployment and a failure point. The app is static hosting plus Supabase.
- **TanStack Query** handles caching, loading and error states, and optimistic mutations with rollback. Tasks load in a single request that embeds assignees, labels and comment counts.
- **The activity log is written by Postgres triggers**, not by the client. Every write path (drag, edit, assignment, label, comment) is recorded the same way, and clients have no insert permission, so history can't be forged.
- **Realtime:** the app subscribes to Postgres changes filtered to the current user. Supabase applies RLS to these events too. Bursts are debounced and held back while a drag is in progress.

Loading, empty and error states

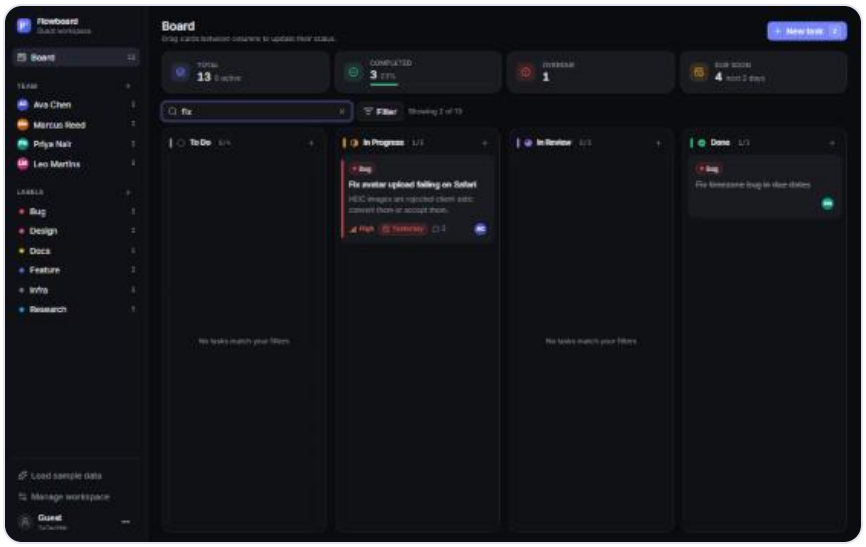
- Splash while the guest session is created; skeleton columns and cards while data loads.
- A welcome banner on an empty board, with "Create task" and one-click **sample data**.
- Per-column empty states that change during a drag ("Drop here...") and when filters hide everything.
- Full-page and inline error states with **Try again** (session, board, comments, activity).
- Toasts for failed mutations, with automatic rollback of optimistic changes.
- Friendly messages for common setup issues (anonymous sign-ins disabled, network down, missing env vars).



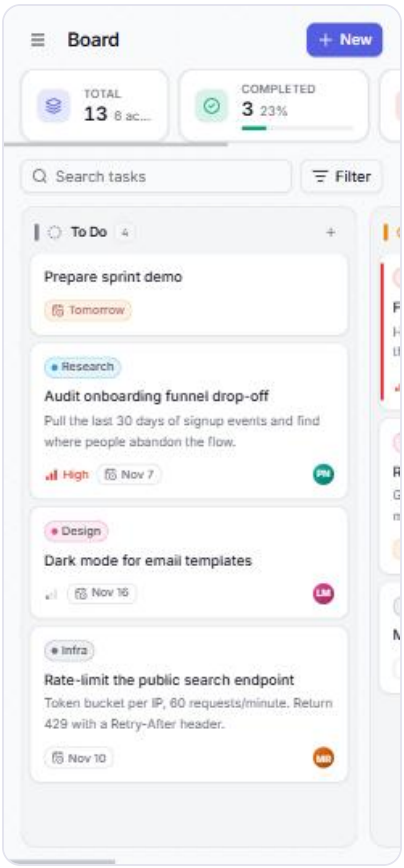
Task detail panel with inline-editable fields and the trigger-generated activity timeline.



Light theme, using the same tokens with light values.



Search narrows every column, shows "x/y" counts, and gives each empty column its own message.



Phone layout: columns snap one at a time and the sidebar moves into a drawer.

03 Features

FEATURE	HOW IT WORKS	
Kanban board (required)	Four default columns; drag between and within columns; status and position persist on drop; updates appear live in other open tabs.	Built
Guest accounts (required)	`signInAnonymously()` on first launch; the session persists in localStorage; every row stores `user_id` (defaulting to `auth.uid()`); RLS on every table. "Start as a new guest" in the account menu demonstrates isolation.	Built
Create tasks (required)	Dialog (or `C` shortcut, or a column's +) with title, description, status, priority, due date, assignees and labels; validation; "Create more" mode; `Ctrl+Enter` to submit.	Built
1. Team & assignees	Add members (name + color swatch, initials avatar). Multiple assignees per task via a `task_assignees` join table. Avatar stacks on cards; the sidebar lists the team with open-task counts and filters on click.	Built

2. Comments	Detail panel with chronological comments, relative and exact timestamps, optimistic posting and delete. Stored in a separate `comments` table; the comment count shows on cards.	Built
3. Activity log	Triggers on `tasks`, `task_assignees`, `task_labels` and `comments` write to `task_activity`. The timeline reads "Moved from To Do → In Review · 2 hours ago", "Assigned Ava Chen", "Changed priority from Normal to High".	Built
4. Labels	Custom colored labels (unique per user), several per task, created inline from the picker or in Workspace settings; filter from the toolbar or sidebar.	Built
5. Due-date indicators	Overdue: red badge plus a red accent bar on the card. Today and next 2 days: amber badges ("Today", "Tomorrow", weekday). Later dates are muted. Done tasks never show as overdue.	Built
6. Search & filtering	Search across title and description (`/` to focus); filters for priority, assignee (including unassigned), label, and overdue / due soon; removable chips and "Showing x of y".	Built
7. Board summary	Header strip: total and active, completed with a progress bar, overdue, and due soon. The overdue and due-soon tiles are toggle filters.	Built

04 Database schema

Seven tables, all with RLS enabled. The full SQL is in the appendix and in the repository at `supabase/schema.sql`; it is idempotent and can be re-run safely.

TABLE	COLUMNS
`tasks`	`id uuid pk`, `title text not null`, `description text`, `status text` (todo · in_progress · in_review · done), `priority text` (low · normal · high), `due_date date`, `position float8`, `assignee_id uuid → team_members`, `user_id uuid → auth.users`, `created_at`, `updated_at`
`team_members`	`id`, `user_id`, `name`, `color` (hex, checked), `avatar_url`, `created_at`
`labels`	`id`, `user_id`, `name` (unique per user), `color`, `created_at`
`task_assignees`	`(task_id, member_id)` pk, `user_id`, `created_at`: many-to-many assignees
`task_labels`	`(task_id, label_id)` pk, `user_id`, `created_at`: many-to-many labels
`comments`	`id`, `task_id → tasks`, `user_id`, `body`, `created_at`
`task_activity`	`id`, `task_id → tasks`, `user_id`, `type`, `payload jsonb`, `created_at`: append-only, trigger-written

Check constraints enforce valid statuses, priorities, colors and text lengths. Foreign keys cascade on delete. Indexes cover `(user_id, status, position)` and the per-task lookups for comments and activity. The single `assignee_id` column from the brief is kept; the multi-assignee feature uses the join table.

05 Security

- **RLS on every table.** Policies compare `user_id` to `auth.uid()` for select, insert, update and delete. Insert and update checks stop a user writing rows owned by someone else.
- **Join tables check both sides.** You can only link *your* task to *your* member or label, and a task's `assignee_id` must be one of your members.
- **Tamper-proof history.** Clients can only read `task_activity`. Triggers write it through a locked-down `SECURITY DEFINER` function with a fixed `search_path`, and `EXECUTE` is revoked from client roles.
- **Least privilege.** All table privileges are revoked from the `anon` role; guests act as `authenticated` after anonymous sign-in.
- **Keys.** Only the public publishable (anon) key is used in the frontend. It is supplied through `.env.local`, which is gitignored; the service-role key is never used anywhere.

Verified end to end. `scripts/verify-rls.mjs` signs in two separate anonymous guests against the live project and runs 19 checks: B cannot read any of A's tasks, members, labels, comments or activity (even by exact id); B's update and delete of A's task affect 0 rows; B cannot comment on A's task, spoof `user_id`, attach A's member or label, or forge activity; unauthenticated requests read nothing. All 19 pass.

06 Running locally

1. Create a free Supabase project. In **SQL Editor**, run the contents of `supabase/schema.sql`.
2. Enable **Authentication → Sign In / Providers → Allow anonymous sign-ins**.
3. Clone the repo and configure environment variables:

```
git clone https://github.com/MonklyS/flowboard.git
cd flowboard
cp .env.example .env.local
# VITE_SUPABASE_URL=https://<project-ref>.supabase.co
# VITE_SUPABASE_ANON_KEY=<anon or sb_publishable_ key>
npm install
npm run dev            # http://localhost:5173
npm run build          # type-check + production build to dist/
node --env-file=.env.local scripts/verify-rls.mjs   # optional RLS test
```

Deployment: `npm run build`, then upload `dist/` to any static host. The live demo uses Cloudflare Pages (`wrangler pages deploy dist`).

07 Tradeoffs and what I'd improve with more time

- **Guest data lives with the browser.** Clearing site data or switching devices loses access to that guest's board. Next step: let a guest upgrade to an email or OAuth account; Supabase can link an identity to an existing anonymous user so the data carries over.
- **Abuse protection.** Anonymous sign-ins can be scripted. In production I'd add Cloudflare Turnstile on sign-in, rely on Supabase's auth rate limits, and run a scheduled job to delete stale anonymous users and their data.
- **Fractional ordering.** Repeated midpoints eventually run out of float precision. Rare in practice, but a periodic rebalance, or switching to string keys (LexoRank-style), would remove the edge case.
- **Realtime granularity.** Change events trigger a debounced refetch of the board. That is simple and always consistent, but large boards would benefit from patching individual rows from the event payload.
- **Team members are not users.** They are names and colors inside one guest's workspace, so comments are always by "You". Real collaboration needs shared workspaces with membership-based RLS policies.
- **Testing.** Coverage today is the RLS integration script plus manual testing in the browser (drag and drop, create, edit, comments, filters, themes, mobile). I'd add Vitest unit tests (filters, ordering, due-date logic) and Playwright end-to-end tests, including a scripted drag.
- **Bundle size.** About 160 KB gzipped in one chunk. Lazy-loading the task panel, calendar and command menu would cut first-load JavaScript.
- **Polish backlog.** Undo for deletes and moves, WIP limits per column, custom columns, a list view, and rich text in descriptions and comments.

A Appendix: full SQL schema

Contents of `supabase/schema.sql`.

```
-- ============================================================================
-- Flowboard — Supabase schema
-- Run this whole file once in the Supabase SQL editor (it is idempotent).
--
-- Security model: every row carries the owning user's id (an anonymous
-- "guest" auth user). Row Level Security is enabled on every table and each
-- policy restricts access to rows where user_id = auth.uid(). Join tables also
-- verify that both sides of the relation belong to the same user.
-- ============================================================================

create extension if not exists pgcrypto;

-- ----------------------------------------------------------------------------
-- Tables
-- ----------------------------------------------------------------------------

create table if not exists public.team_members (
  id          uuid primary key default gen_random_uuid(),
  user_id     uuid not null default auth.uid() references auth.users (id) on delete cascade,
  name        text not null check (char_length(trim(name)) between 1 and 60),
  color       text not null default '#6366f1' check (color ~ '^#[0-9a-fA-F]{6}$'),
  avatar_url  text,
  created_at  timestamptz not null default now()
);

create table if not exists public.labels (
  id          uuid primary key default gen_random_uuid(),
  user_id     uuid not null default auth.uid() references auth.users (id) on delete cascade,
  name        text not null check (char_length(trim(name)) between 1 and 40),
  color       text not null default '#64748b' check (color ~ '^#[0-9a-fA-F]{6}$'),
  created_at  timestamptz not null default now(),
  unique (user_id, name)
);

create table if not exists public.tasks (
  id           uuid primary key default gen_random_uuid(),
  user_id      uuid not null default auth.uid() references auth.users (id) on delete cascade,
  title        text not null check (char_length(trim(title)) between 1 and 200),
  description  text check (description is null or char_length(description) <= 10000),
  status       text not null default 'todo'
               check (status in ('todo', 'in_progress', 'in_review', 'done')),
  priority     text not null default 'normal'
               check (priority in ('low', 'normal', 'high')),
  due_date     date,
  -- Fractional index used to order cards inside a column.
  position     double precision not null default extract(epoch from now()),
  -- Kept for parity with the brief (single primary assignee); the multi-assignee
  -- feature uses task_assignees below.
  assignee_id  uuid references public.team_members (id) on delete set null,
  created_at   timestamptz not null default now(),
  updated_at   timestamptz not null default now()
);

create table if not exists public.task_assignees (
  task_id     uuid not null references public.tasks (id) on delete cascade,
  member_id   uuid not null references public.team_members (id) on delete cascade,
  user_id     uuid not null default auth.uid() references auth.users (id) on delete cascade,
```

```
  created_at  timestamptz not null default now(),
  primary key (task_id, member_id)
);

create table if not exists public.task_labels (
  task_id     uuid not null references public.tasks (id) on delete cascade,
  label_id    uuid not null references public.labels (id) on delete cascade,
  user_id     uuid not null default auth.uid() references auth.users (id) on delete cascade,
  created_at  timestamptz not null default now(),
  primary key (task_id, label_id)
);

create table if not exists public.comments (
  id          uuid primary key default gen_random_uuid(),
  task_id     uuid not null references public.tasks (id) on delete cascade,
  user_id     uuid not null default auth.uid() references auth.users (id) on delete cascade,
  body        text not null check (char_length(trim(body)) between 1 and 5000),
  created_at  timestamptz not null default now()
);

create table if not exists public.task_activity (
  id          uuid primary key default gen_random_uuid(),
  task_id     uuid not null references public.tasks (id) on delete cascade,
  user_id     uuid not null default auth.uid() references auth.users (id) on delete cascade,
  -- created | status_changed | title_changed | description_changed |
  -- priority_changed | due_date_changed | assignee_added | assignee_removed |
  -- label_added | label_removed | commented
  type        text not null,
  payload     jsonb not null default '{}'::jsonb,
  created_at  timestamptz not null default now()
);

-- ------------------------------------------------------------------------------
-- Indexes
-- ------------------------------------------------------------------------------

create index if not exists tasks_user_status_position_idx on public.tasks (user_id, status, position);
create index if not exists team_members_user_idx        on public.team_members (user_id);
create index if not exists labels_user_idx              on public.labels (user_id);
create index if not exists task_assignees_user_idx      on public.task_assignees (user_id);
create index if not exists task_assignees_member_idx    on public.task_assignees (member_id);
create index if not exists task_labels_user_idx         on public.task_labels (user_id);
create index if not exists task_labels_label_idx        on public.task_labels (label_id);
create index if not exists comments_task_idx            on public.comments (task_id, created_at);
create index if not exists task_activity_task_idx       on public.task_activity (task_id, created_at);

-- ------------------------------------------------------------------------------
-- Row Level Security
-- ------------------------------------------------------------------------------

alter table public.team_members   enable row level security;
alter table public.labels         enable row level security;
alter table public.tasks          enable row level security;
alter table public.task_assignees enable row level security;
alter table public.task_labels    enable row level security;
alter table public.comments       enable row level security;
alter table public.task_activity  enable row level security;

-- Simple owner tables: full CRUD on your own rows only.
do $$
declare t text;
begin
  foreach t in array array['team_members', 'labels', 'tasks'] loop
    execute format('drop policy if exists "own rows: select" on public.%I', t);
    execute format('drop policy if exists "own rows: insert" on public.%I', t);
```

```
    execute format('drop policy if exists "own rows: update" on public.%I', t);
    execute format('drop policy if exists "own rows: delete" on public.%I', t);
    execute format($p$create policy "own rows: select" on public.%I for select to authenticated
                     using (user_id = (select auth.uid()))$p$, t);
    execute format($p$create policy "own rows: insert" on public.%I for insert to authenticated
                     with check (user_id = (select auth.uid()))$p$, t);
    execute format($p$create policy "own rows: update" on public.%I for update to authenticated
                     using (user_id = (select auth.uid())) with check (user_id = (select auth.uid()))$p$, t);
    execute format($p$create policy "own rows: delete" on public.%I for delete to authenticated
                     using (user_id = (select auth.uid()))$p$, t);
  end loop;
end $$;

-- A task's single assignee must be one of your own team members.
drop policy if exists "own rows: insert" on public.tasks;
drop policy if exists "own rows: update" on public.tasks;
create policy "own rows: insert" on public.tasks for insert to authenticated
  with check (
    user_id = (select auth.uid())
    and (assignee_id is null or exists (
      select 1 from public.team_members m where m.id = assignee_id and m.user_id = (select auth.uid())))
  );
create policy "own rows: update" on public.tasks for update to authenticated
  using (user_id = (select auth.uid()))
  with check (
    user_id = (select auth.uid())
    and (assignee_id is null or exists (
      select 1 from public.team_members m where m.id = assignee_id and m.user_id = (select auth.uid())))
  );

-- Join tables: you can only link your own task to your own member/label.
drop policy if exists "own rows: select" on public.task_assignees;
drop policy if exists "own rows: insert" on public.task_assignees;
drop policy if exists "own rows: delete" on public.task_assignees;
create policy "own rows: select" on public.task_assignees for select to authenticated
  using (user_id = (select auth.uid()));
create policy "own rows: insert" on public.task_assignees for insert to authenticated
  with check (
    user_id = (select auth.uid())
    and exists (select 1 from public.tasks t where t.id = task_id and t.user_id = (select auth.uid()))
    and exists (select 1 from public.team_members m where m.id = member_id and m.user_id = (select auth.uid()))
  );
create policy "own rows: delete" on public.task_assignees for delete to authenticated
  using (user_id = (select auth.uid()));

drop policy if exists "own rows: select" on public.task_labels;
drop policy if exists "own rows: insert" on public.task_labels;
drop policy if exists "own rows: delete" on public.task_labels;
create policy "own rows: select" on public.task_labels for select to authenticated
  using (user_id = (select auth.uid()));
create policy "own rows: insert" on public.task_labels for insert to authenticated
  with check (
    user_id = (select auth.uid())
    and exists (select 1 from public.tasks t where t.id = task_id and t.user_id = (select auth.uid()))
    and exists (select 1 from public.labels l where l.id = label_id and l.user_id = (select auth.uid()))
  );
create policy "own rows: delete" on public.task_labels for delete to authenticated
  using (user_id = (select auth.uid()));

-- Comments: read/write/delete your own, only on your own tasks.
drop policy if exists "own rows: select" on public.comments;
drop policy if exists "own rows: insert" on public.comments;
drop policy if exists "own rows: delete" on public.comments;
create policy "own rows: select" on public.comments for select to authenticated
  using (user_id = (select auth.uid()));
```

```
create policy "own rows: insert" on public.comments for insert to authenticated
  with check (
    user_id = (select auth.uid())
    and exists (select 1 from public.tasks t where t.id = task_id and t.user_id = (select auth.uid()))
  );
create policy "own rows: delete" on public.comments for delete to authenticated
  using (user_id = (select auth.uid()));

-- Activity log is append-only and written by triggers; clients can only read it.
drop policy if exists "own rows: select" on public.task_activity;
create policy "own rows: select" on public.task_activity for select to authenticated
  using (user_id = (select auth.uid()));

-- ---------------------------------------------------------------------------
-- Triggers: updated_at + activity log
-- ---------------------------------------------------------------------------

create or replace function public.set_updated_at()
returns trigger language plpgsql set search_path = '' as $$
begin
  new.updated_at := now();
  return new;
end $$;

drop trigger if exists tasks_set_updated_at on public.tasks;
create trigger tasks_set_updated_at before update on public.tasks
  for each row execute function public.set_updated_at();

-- Writes one activity row. SECURITY DEFINER so it can bypass the read-only
-- policy on task_activity; it always stamps the row with the task owner's id
-- and silently skips if the task is already gone (cascading deletes).
create or replace function public.log_task_activity(p_task_id uuid, p_type text, p_payload jsonb)
returns void language plpgsql security definer set search_path = '' as $$
declare v_owner uuid;
begin
  select user_id into v_owner from public.tasks where id = p_task_id;
  if v_owner is null then return; end if;
  insert into public.task_activity (task_id, user_id, type, payload)
  values (p_task_id, v_owner, p_type, coalesce(p_payload, '{}'::jsonb));
exception when foreign_key_violation then
  null;
end $$;
revoke all on function public.log_task_activity(uuid, text, jsonb) from public, anon, authenticated;

create or replace function public.tasks_activity_trigger()
returns trigger language plpgsql security definer set search_path = '' as $$
begin
  if tg_op = 'INSERT' then
    perform public.log_task_activity(new.id, 'created', jsonb_build_object('status', new.status));
    return new;
  end if;

  if new.status is distinct from old.status then
    perform public.log_task_activity(new.id, 'status_changed',
      jsonb_build_object('from', old.status, 'to', new.status));
  end if;
  if new.title is distinct from old.title then
    perform public.log_task_activity(new.id, 'title_changed',
      jsonb_build_object('from', old.title, 'to', new.title));
  end if;
  if new.description is distinct from old.description then
    perform public.log_task_activity(new.id, 'description_changed', '{}'::jsonb);
  end if;
  if new.priority is distinct from old.priority then
    perform public.log_task_activity(new.id, 'priority_changed',
```

```
      jsonb_build_object('from', old.priority, 'to', new.priority));
  end if;
  if new.due_date is distinct from old.due_date then
    perform public.log_task_activity(new.id, 'due_date_changed',
      jsonb_build_object('from', old.due_date, 'to', new.due_date));
  end if;
  return new;
end $$;

drop trigger if exists tasks_activity on public.tasks;
create trigger tasks_activity after insert or update on public.tasks
  for each row execute function public.tasks_activity_trigger();

create or replace function public.task_assignees_activity_trigger()
returns trigger language plpgsql security definer set search_path = '' as $$
declare v_name text;
begin
  if tg_op = 'INSERT' then
    select name into v_name from public.team_members where id = new.member_id;
    perform public.log_task_activity(new.task_id, 'assignee_added',
      jsonb_build_object('member_id', new.member_id, 'name', v_name));
    return new;
  else
    select name into v_name from public.team_members where id = old.member_id;
    perform public.log_task_activity(old.task_id, 'assignee_removed',
      jsonb_build_object('member_id', old.member_id, 'name', v_name));
    return old;
  end if;
end $$;

drop trigger if exists task_assignees_activity on public.task_assignees;
create trigger task_assignees_activity after insert or delete on public.task_assignees
  for each row execute function public.task_assignees_activity_trigger();

create or replace function public.task_labels_activity_trigger()
returns trigger language plpgsql security definer set search_path = '' as $$
declare v_name text; v_color text;
begin
  if tg_op = 'INSERT' then
    select name, color into v_name, v_color from public.labels where id = new.label_id;
    perform public.log_task_activity(new.task_id, 'label_added',
      jsonb_build_object('label_id', new.label_id, 'name', v_name, 'color', v_color));
    return new;
  else
    select name, color into v_name, v_color from public.labels where id = old.label_id;
    perform public.log_task_activity(old.task_id, 'label_removed',
      jsonb_build_object('label_id', old.label_id, 'name', v_name, 'color', v_color));
    return old;
  end if;
end $$;

drop trigger if exists task_labels_activity on public.task_labels;
create trigger task_labels_activity after insert or delete on public.task_labels
  for each row execute function public.task_labels_activity_trigger();

create or replace function public.comments_activity_trigger()
returns trigger language plpgsql security definer set search_path = '' as $$
begin
  perform public.log_task_activity(new.task_id, 'commented',
    jsonb_build_object('comment_id', new.id));
  return new;
end $$;

drop trigger if exists comments_activity on public.comments;
create trigger comments_activity after insert on public.comments
```

```
  for each row execute function public.comments_activity_trigger();

-- Trigger functions are only meant to be called by triggers.
revoke all on function public.tasks_activity_trigger()          from public, anon, authenticated;
revoke all on function public.task_assignees_activity_trigger() from public, anon, authenticated;
revoke all on function public.task_labels_activity_trigger()    from public, anon, authenticated;
revoke all on function public.comments_activity_trigger()       from public, anon, authenticated;

-- ------------------------------------------------------------------------------
-- Grants (anon role gets nothing; guests sign in anonymously and become
-- "authenticated", so RLS above is what scopes their access)
-- ------------------------------------------------------------------------------

revoke all on public.team_members, public.labels, public.tasks, public.task_assignees,
              public.task_labels, public.comments, public.task_activity from anon;
grant select, insert, update, delete on public.team_members, public.labels, public.tasks to authenticated;
grant select, insert, delete on public.task_assignees, public.task_labels, public.comments to authenticated;
grant select on public.task_activity to authenticated;

-- ------------------------------------------------------------------------------
-- Realtime: stream row changes (still filtered by RLS) so open tabs stay in sync
-- ------------------------------------------------------------------------------

do $$
declare t text;
begin
  foreach t in array array['tasks', 'task_assignees', 'task_labels', 'team_members',
                           'labels', 'comments', 'task_activity'] loop
    if not exists (
      select 1 from pg_publication_tables
      where pubname = 'supabase_realtime' and schemaname = 'public' and tablename = t
    ) then
      execute format('alter publication supabase_realtime add table public.%I', t);
    end if;
  end loop;
end $$;

-- DELETE events need the full old row so the client knows which task changed.
alter table public.task_assignees replica identity full;
alter table public.task_labels    replica identity full;
```